Assignment 3

Table of Contents

- Assignment 3
  - Table of Contents
  - Question 1
  - Question 2
  - Question 3
  - Question 4
  - Question 5
  - Question 6
  - Question 7
  - Question 8
  - Question 9
  - Question 10
  - Question 11
  - Question 12
  - Question 13
    - a)
    - b)
    - c)
    - d)
  - Question 14
    - a)
      - Special Cases :
    - b)
      - Time Complexity :
    - c)
    - d)
    - e)
  - Question 15
  - Question 16
    - a)
    - b)

- c)
- d)
- Question 17
  - a)
  - b)
  - c)
  - d)
  - Question 18
- Question 19
- Question 20

Question 1

Example Matrix :

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 0 & 0 & 7 & 8 \\ 9 & 0 & 10 & 0 & 11 \\ 12 & 13 & 0 & 0 & 14 \\ 15 & 0 & 0 & 0 & 16 \end{bmatrix}$$

In an $n \times n$ matrix, non zero elements are:

- n first row elements
- n first column elements
- n last column elements
- n anti diagonal elements

In this counting few elements are repeated like $(1,1),(1,n)[twice],(n,1)$

> $\therefore$ Total non zero elements in ***a-matrix*** are $4n-4$

As there are only $4n-4$ non zero elements , we can make an algorithm which only traverse through these elements and add them.
We can achieve $O(n)$ time complexity by adding the elements as a row, column and anti-diagonal.

C++ Implementation

```
#include <vector>
using namespace std;

vector<vector<int>> addAmatrix(vector<vector<int>>& a, vector<vector<int>>& b ) {
    size_t n = a.size();
    vector<vector<int>> c ;
    //adding anti
    for(int i = 1 ; i < n ; i++) {
        c[i][n-i-1] = a[i][n-i-1] + b[i][n-i-1] ;
    }
    //adding 1st column & last column
    for(int i = 0 ; i < n ; i++) {

        c[i][n-1] = a[i][n-1] + b[i][n-1] ;
        if(i == n-1) break ;
        c[i][0] = a[i][0] + b[i][0] ;
    }
    //adding 1st row
    for(int j = 1 ; j < n-1 ; j++) {
        c[0][j] = a[0][j] + b[0][j];
    }
    return c ;
}
```

Question 2

```
#include <bits/stdc++.h>
using namesapce std;

bool isIdempotent(vector<vector<int>>& mat) {
    int n = mat.size() ;

    for(int i = 0 ; i < n ; i++) {
        for(int j = 0 ; j < n ; j++) {
            int sum = 0 ;

            for(int k = 0 ; k < n ; k++) {
                sum += mat[i][k] * mat [k][j]
            }

            if(sum!=mat[i][j]) return false ;
        }
    }

    return true;
}

int main() {
   int n;
   cin>>n;
   vector<vector<int>> mat(n , vector<int> (n)) ;
   for(int i = 0 ; i < n ; i++) {
    for(int j = 0 ; j < n ; j++) {
        cin>>mat[i][j];
    }
   }
   if(isIdempotent(mat)) cout<<"Yes, given matrix is idempotent."<<endl ;
   else cout<<"Not Idempotent";

}
```

Question 3

```
#include <bits/stdc++.h>
//used <iostream> , <vector>
using namespace std;

int main() {
    int row , col ;
    cin>>row>>col ;
    vector<vector<int>> mat(row , vector<int> (col)) ;
    int maxVal = INT_MIN ;
    for(int i = 0 ; i < row ; i++) {
        for(int j = 0 ; j < col ; j++) {
            cin>>mat[i][j] ;
            if(mat[i][j] > maxVal) mat[i][j] = maxVal ;
        }
    }
    cout<<maxVal<<endl ;
    return 0;
}
```

Question 4

- 90 Degrees Clockwise:
  - First, we transpose the matrix (swap rows with columns).
  - Second, we reverse each individual row.
- 180 Degrees
  - Flipping the matrix upside down (reversing the order of the rows), and
  - Then flipping it left-to-right (reversing the elements inside each row).

```cpp
#include <iostrea>
#include <vector>
#include <algorithm>
#include <bits/stdc++.h>
using namespace std ;

void rotate90(vector<vector<int>>& mat) {
    int n = mat.size();

    transposing matrix
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            swap(mat[i][j], mat[j][i]);
        }
    }
    // reversing each row
    for (int i = 0; i < n; i++) {
        reverse(mat[i].begin(), mat[i].end());
    }
}

void rotate180(vector<vector<int>>& mat) {
    int n = mat.size();

    // reversing the order
    reverse(mat.begin(), mat.end());

    // reversing each row
    for (int i = 0; i < n; i++) {
        reverse(mat[i].begin(), mat[i].end());
    }
}
```

Question 5

A list in STL is a contiguous container that allows the inserting and erasing of elements in constant time and iterating in both directions.

- Characteristics
    - Can be traversed in both directions
    - Can't randomly access elements

- Elements are not contigous i.e. scattered in memory, linked together by pointers .
- Each node contains two pointers which makes it bidirectional.

Question 6

```
#include <iostream>
using namespace std ;

int main() {
    int c[] = {2.8,3.4,4,6.7,5} ;
    int j , *p = c , *q = c ;
    for(int i = 0; i < 5 ; i++) {
        cout<<*q ;
        ++q ;
    }
    for(int j = 0 ; j < 5 ; j++) {
        cout << *p ;
        ++p;
    }
}
```

On runnign this code in a compiler it gives error of decimal truncation.

But if I assume the conversion of doubles into integer, output will be `2346523465`

Question 7

```
#include <iostream>
using namespace std ;

class Node {
public:
    int data;     // Data value
    Node* next;   // Pointer to next node

    Node(int data1, Node* next1) {
        data = data1;
        next = next1;
    }

    Node(int data1) {
        data = data1;
        next = nullptr;
    }
    Node() {
        data = 0 ;
        next = nullptr ;
    }
};

Node* reverseNll(Node* head, int k) {
    if (!head) return NULL;

    Node* temp = head;
    Node* front = NULL;
    Node* prev = NULL;
    int cnt = 0;
    while (temp != NULL && cnt < k)  {
        front = temp->front;
        temp->front = prev;
        prev = temp;
        temp = front;
        cnt++;
    }
}
```

Question 8

```
#include <iostream>
struct lnkdlist
{
    int val;
    lnkdlist *next;
};
bool isPrime (long long a)
{
    if (a<2)
        return false;
    for (int i = 2; i*i<=a; i++)
        if (a%i==0)
            return false;
    return true;
}
void buildList (lnkdlist *&head, lnkdlist *&tail, int len)
{
    for (int i = 0; i<len; i++)
    {
        lnkdlist *newNode = new lnkdlist;
        std::cin >> newNode->val;
        newNode->next = nullptr;
        if (head)
        {
            tail->next = newNode;
            tail = newNode;
        }
        else
        {
            head = newNode;
            tail = newNode;
        }
    }
}
void mergeToFirstPrime (lnkdlist *head1, lnkdlist *head2, lnkdlist *&tail1)
{
    lnkdlist *t1 = head1, *t2 = head2;
    for (int i = 1; t2; i++)
    {
        if (isPrime(i+1))
```

```
        {
            lnkdlist *temp = t2;
            t2 = t2->next;
            temp->next = t1->next;
            t1->next = temp;
            t1 = t1->next;
        }
        tail1 = t1;
        t1 = t1->next;
    }
    while (t1)
    {
        tail1 = t1;
        t1 = t1->next;
    }
}
int main()
{
    int lenA, lenB;
    std::cin >> lenA >> lenB;
    lnkdlist *headA = nullptr, *tailA = nullptr;
    buildList(headA, tailA, lenA);
    lnkdlist *headB = nullptr, *tailB = nullptr;
    buildList(headB, tailB, lenB);
    mergeToFirstPrime(headA, headB, tailA);
    lnkdlist *traverser = headA;
    while (traverser)
    {
        std::cout << traverser->val << " ";
        traverser = traverser->next;
    }
    std::cout << "\n";
    //destructor starts here
    lnkdlist *next;
    traverser = headA;
    while (traverser)
    {
        next = traverser->next;
        delete traverser;
        traverser = next;
    }
}
```

Question 9

```
#include <iostream>
#include <list>
using namespace std;

void oddify(list<T>&A) {
    auto it_A = A.begin();
    for(int i = 1 ; it_A != A.end() ; i++ , it_A++) {
        if(i%2){
            continue;
        }
        else{
            A.erase(it_A) ;
        }
    }
    it_A = A.begin() ;
    while(it_A!=A.end()){
        cout<<*it_A<<" ";
        it_A++;
    }
}
```

Question 10

Question 11

To find the intersectional node if it exists, we can simply traverse through the linked list A by $(m$ - $n)$ steps, after which we can traverse through both the linked lists simultaneously. This will guarantee us the answer as, if the linked lists have a common node, they will have the same tail at the end and the last few nodes upto the first common node will be the same.s

The time complexity analysis gives us the worst case scenario when there are no common nodes or the last node is common, where in both we would have to traverse through A once and hence the time complexity of this algortihm is:

$$O\,(m)$$

Question 12

Pseudo Code

Function M(A,B) :-

```
list C //initialize
a<-A.head
b<-B.head
if *a<=*b :
    C.head<-a
    a<-a.next
else :
    C.head<-b
    b<-b.next

prev<-C.head
while(a!=NULL&&b!=NULL) :
    if *a <= *b :
        prev->next <- a
        prev<- a
        a<- a.next
    else :
        prev->next <-b
        prev<-b
        b<-b.next
    while(a!=NULL)
        prev->next <- a
        prev <- a
        a <- a.next
    while(b!=NULL)
        prev->next <- b
        prev <- b
        b <- b.next
```

The space complexity is $O(1)$ because only a constant number of extra pointers are being used. The merge is stable because elements in either list maintain their order (because the iterators a and b move ahead only when the elements pointed to by them are inserted in list).

Question 13

a)

Create 2 temporay lists, one which stores all the nodes with data < x and the other stores all the nodes with data >= x.(Here, we only need to traverse the input linked list once - check the data of the current node and append the node into the list corresponding to the ineqaulity obtained).

Merge the two lists (merge the second list at the end of first list) and set the last pointer to NULL.

Time complexity : $O(n)$, as we only traverse the array once.

Space complexity : $O(1)$ - we create and use a constant extra number of pointers to create and traverse through the lists.

b)

Algorithm :
Use an unordered set. Traverse through the linear linked list and add elements to the unordered set, if the length of the unordered set remains same even after we add the element, then it is repeated so remove the node from the linked list.

Time complexity : $O(n)$
Space complexity : $O(n)$

c)

Algorithm :
To find the middle element in a single pass, we can use two pointers - slow and fast. The slow pointer increments/traverses the list one by one while on the other hand the fast pointer increments the list by skipping one elements i.e. the fast pointer moves two nodes ahead.

d)

To detect a loop in a singly linked list, we can use two pointers - slow and fast. The slow pointer traverses the list one by one while the fast pointer traverses the list by moving two nodes ahead. Also, after each increment, check if nodes are equal or not. If at any point, any of the pointer = NULL, then stop iterating.

To remove the loop, we need to identify the intersection node i.e. the node at which the loop closes. First, we will also use 2 other pointers $prev$ (which points at the node previous to the node pointed by $slow$ pointer) and $head$ (which initially points at the head node of the linked list).
Now, after we detect a loop, we know that the $slow$ and $fast$ both pointers are pointing at the same node, while the $prev$ is pointing at the node before the meetup node.
Now, move both $head$ and $slow$ pointer one by one (the $prev$ pointer is also incremented accordingly) till they both point at the same node. The node at which they both point is the intersection node and the $prev$ pointer is pointing at the last node of the linked list.

Now, free the prev$\rightarrow$next (i.e. prev$\rightarrow$next == NULL)so as to remove the loop.

Question 14

a)

To solve this we can create 2 n-sized vectors, one to keep track of maximum value present in the ith column and the other to keep track of the minimum value present in the jth row. After getting the values, we simply go over the matrix once again and for each value we check whether its equal to both the current row's minimum value and the current column's maximum value. If it is equal, we can simply print the position and value of the found position.

A minor optimisation could be that if we can skip the inner loop such if the current row's minimum is not equal to the maximum of the current column.

<u>Special Cases</u> :

- The case where saddle points *may not exist* are when none of the values from one of the precomputed arrays is equal to the other.
- There can also be cases of multiple values from a row or a column be a saddle point if a row or column has multiple common minimums or maximums and the corresponding column or row also has that as its maximum or minimum. $eg-$

$$\begin{bmatrix} 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 \end{bmatrix}$$

For $n = 5$ and $m = 4$.

b)

To rotate an $n \times n$ matrix by 90°, we can simply traverse over the first quadrant of the matrix for even n and the first quadrant in addition to the extra half-column common to both for odd n. Both of these cases can simply be handled by having the outer loop iterate until its $less\ than\ \frac{n}{2}$ and the inner loop iterate until its $less\ than\ \frac{n+1}{2}$.

<u>Note</u> : Both the indexes for odd n don't require us to go for $\frac{n+1}{2}$ as there will be double counting otherwise, and for even n, $\frac{n}{2}$ and $\frac{n+1}{2}$ are same due to integer-division.

To rotate the matrix without using any extra-space (in-place), we can do the following swaps:

- First we swap the values of $a[n - 1 - j][i]$ and $a[i][j]$.
- Next we swap the values of $a[n - 1 - i][n - 1 - j]$ and $a[n - 1 - j][i]$.
- Next we swap the values of $a[j][n - 1 - i]$ and $a[n - 1 - i][n - 1 - j]$.
- Finally, we can just swap the values of $a[i][j]$ and $a[j][n - 1 - i]$.

The challenge posed by in-place rotation isn't much as even if we had used extra space to store the initial value of $a[i][j]$ into a $temp$ variable, we would have had to finally put that value into $a[j][n - 1 - i]$.

To prove the correctness of this algortihm, we can show that this works for both even and odd n (As shown earlier) and that it actually handles the 90° correctly.

To prove that the indexes are handled correctly, we can simply take an example with $n = 5$, $i = 1$, and $j = 2$. If we calculate all the other indexes correctly, we find that we are accessing the values present in $a[1][2]$ (the base case) and $a[2][1]$, $a[3][2]$, and $a[2][3]$. Which when we actually manually superimpose the image of the matrix over itself when rotated by 90°, we get the same indexes being handled corresponding to $a[1][2]$.

<u>Time Complexity</u> :

The time complexity of the program is $O(n^2)$ as we are iterating over the first quadrant of the matrix, so when the nuber of elements are n^2, we iterate over roughly $\frac{n^2}{4}$ elements and thus our time complexity is:

$$O(n^2)$$

c)

For this we can use a recursive function to traverse through the matrix that only goes ahead if there is a higher value ahead of it else it move one step back and simultaneously counts the number of moves we had made to reach the current position. Now rather than further exaplaining, I am attaching the recursive function along with the padding itself:

```
#include <iostream>
#include <vector>
using namespace std;
long long formingTable(const vector<vector<int>> &a, vector<vector<long long>> &lookUpTable, int
{
    if (lookUpTable[x][y] != 0)
    {
        return lookUpTable[x][y];
    }
    static const long long dx[4] = {-1, 0, 0, 1}, dy[4] = {0, -1, 1, 0};
    long long maxLen = 1;
    for (int i = 0; i < 4; i++)
    {
        if ((x + dx[i]) >= 0 && (x + dx[i]) < n && (y + dy[i]) >= 0 && (y + dy[i]) < m && a[x][y
        {
            maxLen = max(maxLen, 1 + formingTable(a, lookUpTable, x + dx[i], y + dy[i], n, m));
        }
    }
    lookUpTable[x][y] = maxLen;
    return maxLen;
}
long long maxLength(const vector<vector<int>> &a, const int &n, const int &m)
{
    vector<vector<long long>> lookUpTable(n, vector<long long>(m, 0));
    long long maxPath = 0;
    for (int i = 0; i<n; i++)
    {
        for (int j = 0; j<m; j++)
        {
            maxPath = max(formingTable(a, lookUpTable, i, j, n, m), maxPath);
        }
    }
    return maxPath;
}
int main()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
```

```
            cin >> a[i][j];
        }
    }
    cout << "The longest path length is : " << maxLength(a, n, m) << "\n";
}
```

d)

e)

To solve this question, rather than going through all the indexes and forming an $O(n^3)$ solution, we try formulate a different approach. We first observe that if we intially take 2 matrices $A_{n\times k}$ and $B_{k\times m}$, with A having k_1 and B having k_2 elements in them and multiply them to get $C_{n\times m}$, we get C_{ij} in the form of:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} \times B_{kj}$$

Now after this observation, we can create an efficient form to access the two arrays if not already available like a linked list representation which stores the sparse matrices in an orderly manner. Now if we start our iteration with A_{ik}, we notice that each element in the matrix can be multiplied with at most k_2 values and thus we only need to find whether the pairs actually form or not. We check through each element in A against all k_2 elements in B and adding them to an approriate data structure such as `unordered_map` or someother one which we are using to store the multiplied matrix C. If the pair is appropriate, we simply add the product as each C_{ij} is made up of a sum n products. Hence, we can also conclude the time complexity of the algorithm being:

$$O(k_1 \cdot k_2)$$

Question 15

Pseudo Code

```
Double linked list  L , Queue Q
it <- L.head
while True :
    if it  -> child != NULL :
        Q.enqueue(it->child)
    if it -> next == NULL :
        if Q.empty() :
            break
        else :
            it -> next = Q.front()
            Q.dequeue()
    it <- (it -> next)
```

Question 16

a)

To solve this we can do the following things:

- We maintain a running sum variable that keeps track of the cumulative sum $cSum$ as well as another variable that stores the most-negative running sum $negSum$ upto the current l_i (default inititalized to 0), as well as the $start$, $potentialStart$, end, $potentialEnd$ node to actually store the values where the segment's start and end as well as a maximum sum variable $maxSum$.
- At any node (let's name it i), we check whether adding the number will make the $cSum$ the most-negative running sum or not. If it does not, then we simply check whether $(cSum$ - $negSum) > max$ or not. If it does, then we update $end = potentialEnd$ and $start = potentialStart$, otherwise we do nothing. To account for all negative values in the list, we also add the condition after the above sequence that, if the current l_i is greater than the max after doing the above steps, we update $max = l_i$ and $start = end = potentialEnd$.
- However if $cSum$ did turn out to be the current $negSum$, we update $negSum = cSum$ and $potentialStart = potentialEnd \rightarrow next$.
- Our update after each node would be to move the current 'traversing' pointer to the next node which is simply our $potentialEnd$.

b)

This problem is similar to 13) d) and we initially follow the exact same steps till we get both $pointer \rightarrow next$ as equal and when there is no loop (we find a $nullptr$ from the faster pointer), we

simply return 0. But after this we:

- Start maintaing a counter $length$ (initialised to 0) that will actually maintain the length of the loop.
- We start traversing through the linked list using the pointer which points to the element that is just before the startring point of the loop in single node jumps while also incrementing the $length$ variable. We do this until both the pointers point to the same node and thus we have now the count of the nodes in the loop of the linked list.
- We can now simply return $length$.

c)

To solve this problem we can do the following steps, while maintaning 2 pointers, namely p, q and $temp$:

- We traverse to the second last node using q, and extract the last node in $temp$.
- We insert $temp$ right after p.
- We update p to just after the newly inserted node (Just before the position where the next node is to be inserted).
- We repeat the above steps up $\frac{n}{2}$ times.

d)

To solve this problem, we can leverage the previously written algorithm to merge 2 sorted linked lists.

- We first pair-up the linked lists and merge them just like we would merge 2 sorted linked lists, resulting in us now having $\lceil \frac{k}{2} \rceil$.
- We then merge the next $\lceil \frac{k}{2} \rceil$ lists to get $\lceil \frac{k}{4} \rceil$ lists.
- We repeat this step until we have get back a single merged linked list.

We can handle the pairing of the linked lists using a deque, where we insert k pointers and keep picking the first 2 pointers and merging them and inserting the $head$ of the merged linked lists from the back.

Question 17

a)

Create a dummy node and point it ot the head of the list.
Point L.head to this dummy node.
Set a 'prev' pointer to this dummy node.
Define a function swap which takes in a node pointers as follows :

```
swap(prev,second):
    *((*prev).next).next = (*second).next
    second->next = prev->next
    prev->next = second
Initialize 'second' to prev -> next -> next
while second->next != NULL and second != NULL ,
apply swap(prev,second) and then advance both prev and next by two positions each, i.e. prev = s
Repeat, Delete dummy node after loop breaks and readjust L.head
```

Time Complexity is $O(n)$ as only one pass over the linked list occurs

b)

PseudoCode

odd_first ← NULL
it ← L.head
while(it.next!=NULL)
    it = it → next
tail← it
it = L.head
Node dummy //create a dummy node
L.head = &dummy
dummy.next = it
prev ← L.head

while(it!=odd_first) :
    if(!odd_first) : odd
    if it→data % 2 == 1 :
        tail→next = it

tail = it

prev->next = it -> next

it = it -> next

tail -> next = NULL

else :

prev = prev $\rightarrow$ next

it = it $\rightarrow$ next

$O(n)$ $\rightarrow$ one pass

c)

```
swap(L,i,j)
a<-L.head
b<-L.head
    for i from 1 to i - 2 :
        a = a -> next
    for i from 1 to j - 2 :
        b = b -> next
        temp <- (a->next->next)
        a->next->next = b->next->next
        b->next->next = temp
        temp = a->next
        a->next=b->next
        b->next=temp
```

d)

1. Create a dummy node tt head of list
2. Initialize two pointers , 'boundary' $\leftarrow$ &dummy and 'scanner' $\leftarrow$ *(dummy.next).next
3. While scanner is not `NULL` , keep checking whether scanner->data < x,if yes , swap *(boundary->next) and *scanner. Move boundary and scanner by one node each. If not, move scanner ahead one node
4. Delete the dummy node

Question 18

Considering that the initial list is not a sorted list, $O(n)$ time is the most straightforward way of finding the number of occurences of a value in a given list.

- We initialise a $count$ variable to 0 and a $curr$ pointer to traverse through the list.
- At any node, we simply check whether $curr \rightarrow val$ is equal to v or not, if it is we simply add 1 to count else we do nothing.
- We update $curr$ to the next node and repeat the previous step until it poiints to $nullptr$.

Question 19

Same as question 11

To solve this problem let us first consider 2 linked lists A and B, with lengths m and n. If we consider $m \geq n$ and carry out the following steps:

- We create 3 pointers $currA$ and $currB$ to traverse through A and B, as well as the pointer which returns the frist common node as $comm$ which is initialised to $nullptr$.
- We traverse through $(m$ - $n)$ nodes in A such that we are $(n$ - $1)$ nodes before the end.
- Now we start updating both the pointers simultaneously and check the equality at each step. If we find the equality to hold, we update $comm = currA$ and we break the loop and return it. If we traverse through the entire list and find no common node, out $comm$ is still $nullptr$, so we can easily output that there was no common node using that.

Question 20

PseudoCode

```
it ← L.head
scanner ← it →next
while it → next != NULL :
    if it → a == scanner → b && it → b == scanner → a
        print (a,b) , (b,a)
    scanner = scanner → next
it = it → next
```

$\frac{n(n+1)}{2}$ operations

> $O(n^2)$